

Lesson 17 — ML Pipelines & Feature Engineering

This is an important practical lesson because now we start connecting the individual ML concepts you've learned into a proper real-world workflow.

So far you've done things separately:

```
Data
↓
Scaling
↓
Model
↓
Prediction
```

But in real ML projects, we usually want everything organized into a pipeline.

1. What is Feature Engineering?

Feature engineering means creating, transforming, or selecting features so that a machine-learning model can learn better.

Suppose you have:

```
age
salary
hours_worked
```

You could create:

```
salary_per_hour
```

or:

```
age_group
```

or:

```
hours_per_day
```

These newly created features may contain useful information for the model.

Simple definition:

Feature engineering = converting raw data into useful input features for ML.

2. Real-World Example

Imagine SentinelAI receives telemetry:

```
CPU usage
```

```
Memory usage
Latency
Error rate
Request count
```

Instead of giving only raw values to the model, we could create:

```
CPU usage
Memory usage
Latency
Error rate
    ↓
Feature Engineering
    ↓
CPU-memory ratio
Latency trend
Error rate change
Average latency
Rolling CPU average
    ↓
ML Model
    ↓
Failure probability
```

This is extremely relevant to your SentinelAI project.

3. What is a Pipeline?

A pipeline combines multiple ML steps into one workflow.

For example:

```
Raw Data
    ↓
Scaling
    ↓
Model
    ↓
Prediction
```

Instead of manually doing:

```
X_scaled = scaler.fit_transform(X)

model.fit(X_scaled, y)
```

we can create:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

Then:

```
pipeline.fit(X_train, y_train)
```

The pipeline handles the steps in the correct order.

4. Why Do We Need Pipelines?

Imagine you train your model like this:

```
scaler.fit_transform(X_train)
```

Then later you accidentally do:

```
scaler.fit_transform(X_test)
```

That's wrong.

You should do:

```
scaler.transform(X_test)
```

A pipeline helps prevent these kinds of mistakes.

It also makes your ML workflow:

- cleaner
- reproducible
- easier to deploy
- easier to tune
- less prone to data leakage

5. Simple Pipeline

Let's use Iris again.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)

accuracy = pipeline.score(X_test, y_test)

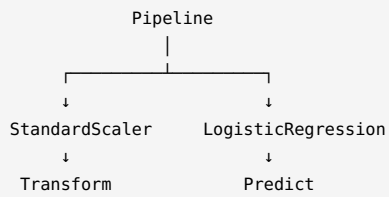
print("Accuracy:", accuracy)
```

6. Understand This Part

```
pipeline = Pipeline([
```

```
("scaler", StandardScaler()),  
("model", LogisticRegression())  
])
```

Think of it as:



The order matters.

First:

```
scaler
```

Then:

```
model
```

7. What Happens During `fit()` ?

When you run:

```
pipeline.fit(X_train, y_train)
```

the pipeline approximately does:

```
X_train  
↓  
StandardScaler  
↓  
Scaled X_train  
↓  
Logistic Regression  
↓  
Learn parameters
```

8. What Happens During `predict()` ?

When you run:

```
pipeline.predict(X_test)
```

the pipeline automatically does:

```
X_test  
↓  
Same fitted scaler  
↓  
Scaled X_test  
↓  
Trained model  
↓
```

```
Prediction
```

You don't have to manually call:

```
scaler.transform(X_test)
```

9. Pipeline + Neural Network

You can also combine a neural network with preprocessing.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", MLPClassifier(
        hidden_layer_sizes=(10,),
        max_iter=1000,
        random_state=42
    ))
])

pipeline.fit(X_train, y_train)

print("Accuracy:", pipeline.score(X_test, y_test))
```

So:

```
Raw Features
  ↓
StandardScaler
  ↓
Neural Network
  ↓
Prediction
```

10. Feature Engineering Example

Let's create a small dataset.

```
import pandas as pd

data = pd.DataFrame({
    "distance": [10, 20, 30, 40],
    "time": [1, 2, 2, 4]
})

print(data)
```

We can create:

```
speed = distance / time
```

Using:

```
data["speed"] = data["distance"] / data["time"]

print(data)
```

Now:

distance	time	speed
10	1	10
20	2	10
30	2	15
40	4	10

We have created a new feature.

That's feature engineering.

11. Another Important Type: Feature Selection

Feature engineering isn't only about creating features.

We can also remove useless features.

Suppose:

100 features

but only:

20 features

are useful.

We can select the important ones.

Why?

Too many irrelevant features can:

- increase complexity
- increase training time
- introduce noise
- contribute to overfitting

This connects directly to Lesson 15.

12. Data Leakage ⚠

This is one of the most important concepts in this lesson.

Suppose you have:

Training data
Testing data

You must not allow information from the test set to influence training.

Wrong:

```
scaler.fit_transform(X)
```

before splitting into train/test.

Because the scaler has now seen the entire dataset.

Correct:

```
X_train, X_test, y_train, y_test = train_test_split(...)

scaler.fit_transform(X_train)

scaler.transform(X_test)
```

Even better:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

This is one reason pipelines are so useful.

13. Pipeline + GridSearchCV

Here's where Lesson 14 and Lesson 17 connect.

You can tune the entire pipeline.

```
from sklearn.model_selection import GridSearchCV

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

param_grid = {
    "model__C": [0.1, 1, 10]
}

grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="accuracy"
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best score:", grid.best_score_)
```

Notice:

```
"model__C"
```

The double underscore:

```
__
```

means:

```
pipeline step → parameter
```

So:

```
model__C
↓
model's C parameter
```

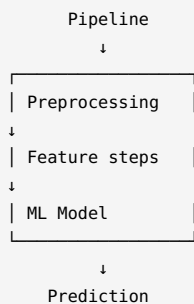
14. Why This Is Powerful

Without a pipeline:

```
Scaling
↓
Feature Engineering
↓
Model
↓
GridSearch
```

You have to manage everything manually.

With a pipeline:



And GridSearchCV can tune the model inside it.

15. SentinelAI Connection 🚀

This lesson is particularly important for your project.

Your SentinelAI pipeline can eventually look like:

```
Prometheus Metrics
↓
Metric Collection
↓
Feature Engineering
↓
Feature Scaling
↓
ML Model
↓
Failure Probability
↓
Severity
↓
Dashboard
```

For example, raw telemetry:

```
CPU = 87%
Memory = 91%
Latency = 420 ms
```

Error rate = 8%

could become features such as:

CPU
Memory
Latency
Error rate
CPU trend
Memory trend
Latency trend
Error rate trend
Rolling averages

Then:

```
graph TD
    Features --> Preprocessing
    Preprocessing --> Model
    Model --> FailureProbability[Failure Probability]
```

That's much closer to a real production ML system.

Lesson 17 Summary

Remember these:

Feature Engineering

Creating or transforming raw data into useful ML features.

Feature Selection

Choosing useful features and removing irrelevant ones.

Pipeline

Combining preprocessing and model steps into one workflow.

Data Leakage

When information that should be unavailable during training influences the model.

Pipeline + GridSearchCV

Allows you to tune a model while keeping preprocessing inside the cross-validation workflow.

The ML workflow you've built so far

You now have:

```
graph TD
    1[1. Understand Data] --> 2[2. Split Data]
    2 --> 3[3. Feature Engineering]
```

```
4. Preprocessing / Scaling
  ↓
5. Choose Model
  ↓
6. Train
  ↓
7. Evaluate
  ↓
8. Tune Hyperparameters
  ↓
9. Prevent Overfitting
  ↓
10. Build Pipeline
  ↓
11. Deploy
```

And we're now very close to the production side of ML.

Next → Lesson 18: Model Deployment & Monitoring

We'll learn how a trained ML model actually becomes an API/service that other applications can send data to, which will directly connect to your FastAPI + SentinelAI backend.